

Big Data Analytics & Predictive Intelligence for E-Commerce

Technologies Used

- Python
- Pandas
- NumPy
- PySpark
- MySQL
- Scikit-learn
- Matplotlib
- Jupyter Notebook
- Power BI

1. Abstract

This project presents an end-to-end **Big Data Analytics and Predictive Intelligence system for e-commerce transaction data**. The objective is to transform raw transactional data into meaningful business insights and develop a machine-learning-based sales forecasting solution.

The project integrates **MySQL for data storage**, **Pandas for data preparation**, **PySpark for scalable data processing**, and **Scikit-learn for predictive modelling**. The data was validated and cleaned before analytical processing. Several business dimensions, including customers, products, payment methods, transaction status, and sales trends, were analysed.

A **Linear Regression model** was subsequently developed using daily sales data to predict future sales. Model performance was evaluated using MAE, MSE, RMSE, and R^2 .

2. Introduction

The rapid growth of e-commerce generates large volumes of transactional data every day. Analysing this data can help businesses understand customer behaviour, monitor revenue, identify high-performing products, and predict future sales.

Traditional data-processing techniques may become inefficient when datasets grow significantly. Big-data technologies such as **Apache Spark** provide scalable processing capabilities, while machine learning can be used to generate predictive insights.

This project combines these technologies into a complete analytics pipeline:

MySQL → Data Validation → Data Cleaning → PySpark Processing → Business Analytics → Machine Learning → Forecasting → Visualization

3. Problem Statement

E-commerce businesses need to understand their transaction data and predict future sales to support better decision-making.

The main problems addressed in this project are:

1. How can raw e-commerce transaction data be cleaned and validated?
2. How can large transactional datasets be processed efficiently?
3. What insights can be extracted from customer and transaction data?
4. How can historical sales data be used to predict future sales?
5. How can analytical results be presented through visualizations and dashboards?

4. Objectives

The major objectives are:

- Store and retrieve transactional data using MySQL.
- Identify and handle data-quality problems.
- Prepare a reliable analytical dataset.
- Process transaction data using PySpark.
- Analyse revenue and transaction behaviour.
- Identify important customer and product patterns.
- Analyse payment methods and transaction status.
- Generate daily sales trends.
- Develop a sales prediction model.
- Evaluate model performance.
- Predict sales for upcoming days.
- Prepare the results for business dashboards.

5. Dataset Description

The dataset contains e-commerce transaction information.

Main attributes

Column	Description
transaction_id	Unique transaction identifier
customer_id	Customer identifier
product_id	Product identifier
transaction_date	Date and time of transaction
quantity	Number of units purchased

Column	Description
unit_price	Price per unit
total_amount	Total transaction amount
discount_applied	Discount information
status	Transaction status
payment_method	Payment method
shipping_cost	Shipping cost

6. Data Storage

The transactional dataset was imported into **MySQL**.

MySQL was used as the structured data-storage layer, allowing the project to demonstrate a realistic data pipeline instead of working only with a CSV file.

The cleaned data was subsequently transferred into Python/PySpark for analytical processing.

7. Data Quality Analysis

During validation, several structurally inconsistent records were identified.

Examples included:

- Invalid product identifiers
- Invalid transaction dates
- Invalid transaction statuses
- Invalid payment methods
- Incorrectly shifted values between columns

The following invalid records were identified:

Field	Invalid Records
product_id	255
transaction_date	125
status	7
payment_method	31

These records indicated structural inconsistencies in part of the source data.

Data-cleaning approach

Instead of guessing missing or shifted values, records failing validation rules were removed.

Validation rules were applied to:

- Product IDs
- Transaction dates
- Status values
- Payment methods
- Numeric transaction fields

This approach prevents fabricated information from affecting the machine-learning model.

8. Cleaned Dataset

After validation and cleaning:

Valid transactions: 242

The valid transaction period was:

January 1, 2023 → January 9, 2023

This finding is important because the original dataset did not provide enough valid monthly observations for meaningful monthly forecasting.

Therefore, the forecasting granularity was changed from **monthly to daily**.

9. Feature Engineering

The transaction date was converted into a timestamp.

Additional analytical features were generated:

- Year
- Month
- Day
- Day of week
- Hour
- Net amount

The following relationship was used:

Net Amount = Total Amount – Shipping Cost

These features make the dataset more suitable for analysis and machine learning.

10. PySpark Processing

PySpark was used to demonstrate distributed data-processing capabilities.

The cleaned Pandas dataset was converted into a Spark DataFrame.

The Spark schema included:

```
transaction_id  
customer_id  
product_id  
transaction_date  
quantity  
unit_price  
total_amount  
discount_applied  
status  
payment_method  
shipping_cost
```

PySpark was then used for aggregation and analytical operations.

11. Exploratory Data Analysis

The project analysed several business dimensions.

11.1 Revenue Analysis

Total transaction revenue was calculated using:

```
SUM(total_amount)
```

Average transaction value was also calculated to understand typical customer spending.

11.2 Transaction Analysis

The total number of valid transactions was calculated after data cleaning.

This provides a reliable measure of the usable transaction population.

11.3 Payment Method Analysis

Transactions were grouped by payment method to determine the contribution of different payment channels to total sales.

Analysed payment methods included:

- Credit Card

- Debit Card
 - PayPal
 - Apple Pay
 - Google Pay
-

11.4 Transaction Status Analysis

Transactions were analysed according to their status:

- Completed
- Pending
- Cancelled
- Refunded

This helps identify the distribution of successful and unsuccessful transactions.

11.5 Customer Analysis

Customers were grouped by total transaction amount.

This allows identification of high-value customers and supports customer segmentation.

11.6 Product Analysis

Products were ranked according to their generated sales.

This helps identify high-performing products and supports inventory and marketing decisions.

12. Sales Trend Analysis

Because the valid dataset contained only one month of data, monthly forecasting was not statistically suitable.

Therefore, transaction-level data was aggregated by **day**.

Daily sales were calculated using:

```
Daily Sales = SUM(total_amount)
```

The resulting daily dataset was used as the input for the forecasting model.

13. Machine Learning Model

A **Linear Regression** model was selected for the initial forecasting task.

Independent Variable

Time Index

Dependent Variable

Daily Sales

The time index represents the chronological position of each observation.

The dataset was divided chronologically:

- **80% Training Data**
- **20% Testing Data**

The data was not shuffled because time-series observations should preserve their chronological order.

14. Model Evaluation

The model was evaluated using four standard metrics.

MAE — Mean Absolute Error

Measures the average absolute difference between actual and predicted values.

MSE — Mean Squared Error

Measures the average squared prediction error.

RMSE — Root Mean Squared Error

The square root of MSE and expressed in the same unit as sales.

R² Score

Measures how well the model explains variation in the target variable.

15. Future Sales Prediction

After training the Linear Regression model, the model was used to predict sales for the **next 7 days**.

The prediction output was stored in:

```
sales_forecast.csv
```

The forecast contains:

Column	Description
day	Future prediction day
predicted_sales	Model-predicted sales

16. Visualization

An **Actual vs Predicted Daily Sales** graph was created using Matplotlib.

The graph compares:

- Actual sales from the testing dataset
- Sales predicted by the Linear Regression model

This visualization makes it easier to understand how closely the model follows the actual sales pattern.

17. Business Insights

The analytical system provides several useful business insights:

1. **Revenue monitoring** can help businesses track overall sales performance.
2. **Payment-method analysis** can identify the most frequently used payment channels.
3. **Transaction-status analysis** can highlight cancelled, pending, and refunded transactions.
4. **Customer-level analysis** can identify high-value customers.
5. **Product-level analysis** can identify high-performing products.
6. **Daily sales analysis** helps identify short-term sales patterns.
7. **Sales forecasting** can support short-term inventory and operational planning.
8. **PySpark processing** provides a scalable foundation for larger datasets.

18. Data Quality Limitation

One of the most important findings of this project was the presence of structural inconsistencies in the source data.

Some records contained values belonging to other fields, such as timestamps appearing in the product identifier column or device-related values appearing in the transaction-date column.

Because the original intended values could not be reliably reconstructed, these records were excluded.

This demonstrates an important real-world data-engineering principle:

It is better to remove unreliable records than to introduce fabricated values into an analytical or predictive system.

19. Project Limitations

The current forecasting model has an important limitation.

The cleaned dataset contains only **nine days of valid transaction dates**. Therefore:

- Monthly forecasting was not possible.
- The model uses daily rather than monthly observations.
- The Linear Regression forecast should be considered a **demonstration/prototype**, not a production forecasting system.
- A longer historical dataset would significantly improve forecasting reliability.

For a production system, several months or years of clean historical data should be collected.

20. Future Scope

The project can be further improved by:

Advanced Forecasting

- ARIMA
- SARIMA
- Prophet
- Random Forest
- XGBoost
- LSTM

Advanced Customer Analytics

- RFM Analysis
- Customer Segmentation
- Customer Lifetime Value

Advanced Product Analytics

- Product recommendation
- Market-basket analysis
- Product demand forecasting

Big Data Expansion

The pipeline can be scaled to millions of transactions using PySpark.

Business Intelligence

The cleaned dataset and forecast results can be connected to **Power BI** to create an interactive dashboard.

21. Requirements

Your `requirements.txt` can contain:

```
pandas<3
numpy
pyspark
pyarrow
scikit-learn
matplotlib
seaborn
sqlalchemy
pymysql
jupyter
```

25. Conclusion

The **Big Data Analytics & Predictive Intelligence for E-Commerce** project demonstrates a complete data analytics and machine-learning workflow.

The project successfully integrates **MySQL, Pandas, PySpark, Scikit-learn, and Matplotlib** to transform transactional data into analytical and predictive outputs.

The project also demonstrates real-world data-quality handling by identifying structurally inconsistent records and excluding unreliable observations rather than introducing artificial values.

Daily sales forecasting was implemented using Linear Regression, and the model was evaluated using MAE, MSE, RMSE, and R^2 . A seven-day future sales forecast was also generated.

Although the current dataset contains only nine days of valid historical data, the developed architecture can be extended to larger datasets with longer historical periods. With additional

data, more advanced forecasting algorithms and interactive Power BI dashboards can be incorporated to create a production-level predictive intelligence platform.